

Objets Personnalisés ([PSCustomObject]) et Tableaux de Bord HTML

Ce document détaille la conception de structures de données métiers sur mesure à l'aide de [PSCustomObject], l'abandon des pratiques textuelles destructrices, l'analyse comparative des structures de données, ainsi que les mécanismes de conversion et de stylisation web permettant de générer des tableaux de bord HTML/CSS réactifs et exploitables en production.

1. Structurer la Donnée en Administration Système ?

En administration système moderne, un script ne se contente pas d'exécuter des commandes à l'aveugle : il interroge l'état d'un parc, extrait des indicateurs clés (CPU, RAM, stockage, disponibilité des services), prend des décisions logiques et produit des livrables exploitables (tableaux de bord, rapports d'audit, alertes automatisées).

La valeur d'un script d'infrastructure dépend directement de la façon dont il véhicule la donnée :

- Si la donnée est émise sous forme de **texte brut**, elle est immédiatement détruite pour l'automatisation.
- Si la donnée est émise sous forme d'**objet structuré**, elle conserve son intelligence, ses types natifs et son interopérabilité avec l'ensemble de l'écosystème PowerShell.

2. L'Approche Non-Structurée :

Pour comprendre la supériorité de [PSCustomObject], examinons les quatre erreurs d'architecture les plus fréquentes en entreprise, leurs conséquences techniques directes et leur résolution immédiate.

Write-Host et du texte plat

Un réflexe fréquent chez l'administrateur débutant consiste à afficher directement les grandeurs mesurées à l'écran via des chaînes de caractères :

APPROCHE DÉCONSEILLÉE : Sortie texte cosmétique non réexploitable

```
function Get-DisqueInfo {  
    $Disque = Get-PSDrive -Name C  
    $LibreGo = [math]::Round($Disque.Free / 1GB, 1)  
    Write-Host "Disque C: $LibreGo Go disponibles" -ForegroundColor Cyan  
}
```

Pourquoi cette approche est une impasse technique ?

1. **Perte irrémédiable du flux** : Write-Host écrit directement dans le flux d'information de la console (flux 6) et contourne le flux de succès (flux 1). Il renvoie \$null à toute variable qui tente de capturer son retour :
\$Resultat = Get-DisqueInfo
\$Resultat -eq \$null # Renvoie TRUE ! Rien n'a été mémorisé.
2. **Incompatibilité avec le pipeline** : Une chaîne affichée ou concaténée ne peut plus être triée par valeur numérique (Sort-Object), ni filtrée par critère opérationnel (Where-Object).
3. **Impossibilité d'automatiser** : Impossible de brancher un envoi d'email automatique ou d'exporter les alertes vers un fichier sans devoir réécrire entièrement le script.

La Solution avec [PSCustomObject] :

On émet un objet métier vivant dans le flux de succès standard :

APPROCHE INDUSTRIELLE : Émission d'un objet structuré réexploitable

```
function Get-DisqueInfo {  
    $Disque = Get-PSDrive -Name C  
    $TailleTotaleGo = [math]::Round(($Disque.Used + $Disque.Free) / 1GB, 1)  
    $EspaceLibreGo = [math]::Round($Disque.Free / 1GB, 1)  
    $RatioLibrePct = [math]::Round(($Disque.Free / ($Disque.Used + $Disque.Free)) * 100, 1)  
  
    [PSCustomObject]@{  
        Lecteur      = $Disque.Name  
        TotalGo      = $TailleTotaleGo  
        LibreGo      = $EspaceLibreGo  
        LibrePct     = $RatioLibrePct  
        Statut       = if ($RatioLibrePct -lt 15) { "ALERTE" } else { "OPTIMAL" }  
        DateControle = (Get-Date -Format "yyyy-MM-dd HH:mm")  
    }  
}
```

La donnée est désormais capturable, filtrable et exportable sans modification :

```
$Bilan = Get-DisqueInfo  
$Bilan | Where-Object Statut -eq "ALERTE"  
$Bilan | Export-Csv -Path "C:\Audit\disque.csv" -NoTypeInfo -Encoding UTF8
```

Export-Csv sur une Table de Hachage (Hashtable)

Une confusion majeure consiste à penser qu'une table de hachage (@{}) est équivalente à une ligne de tableau. Une table de hachage est un conteneur dictionnaire de type clé/valeur en

mémoire, **pas un enregistrement tabulaire**.

Le scénario d'échec :

Tentative d'inventaire avec des tables de hachage classiques

```
$Parc = @(
    @{ Serveur = "SRV-AD01"; RAM_Go = 32; Statut = "OK" },
    @{ Serveur = "SRV-FILE01"; RAM_Go = 16; Statut = "Alerte" }
)
```

```
$Parc | Export-Csv -Path "C:\Audit\inventaire.csv" -NoTypeInfoInformation
```

Le résultat en production (fichier corrompu) :

À l'ouverture du fichier inventaire.csv, vous n'obtenez ni les noms de serveurs, ni la RAM, mais les métadonnées internes du dictionnaire .NET :

```
"IsReadOnly";"IsFixedSize";"IsSynchronized";"Keys";"Values";"SyncRoot";"Count"
"False";"False";"False";"System.Collections.Hashtable+KeyCollection";"System.Collections.Hasht
able+ValueCollection";"System.Object";"3"
"False";"False";"False";"System.Collections.Hashtable+KeyCollection";"System.Collections.Hasht
able+ValueCollection";"System.Object";"3"
```

Export-Csv analyse le type .NET sous-jacent de l'objet. Comme il s'agit d'une instance de System.Collections.Hashtable, la commande extrait les propriétés internes de la classe .NET (comme IsFixedSize ou Count), détruisant ainsi l'intégralité des données métiers.

La Solution avec [PSCustomObject] :

Le simple transtypage [PSCustomObject]@{} transforme le dictionnaire associatif en un véritable enregistrement tabulaire doté de colonnes nommées :

APPROCHE INDUSTRIELLE : Objets personnalisés tabulaires

```
$Parc = @(
    [PSCustomObject]@{ Serveur = "SRV-AD01"; RAM_Go = 32; Statut = "OK" },
    [PSCustomObject]@{ Serveur = "SRV-FILE01"; RAM_Go = 16; Statut = "Alerte" }
)
```

```
$Parc | Export-Csv -Path "C:\Audit\inventaire.csv" -Delimiter ";" -NoTypeInfoInformation -Encoding UTF8
```

Résultat obtenu dans inventaire.csv :

```
"Serveur";"RAM_Go";"Statut"  
"SRV-AD01";"32";"OK"  
"SRV-FILE01";"16";"Alerte"
```

L'éparpillement en variables multiples

En métrologie système, une sonde isolée n'a que peu de sens. L'administrateur a besoin de croiser plusieurs sources :

- L'espace disque (issu de Get-PSDrive ou CIM),
- La consommation mémoire (issue de Get-Process),
- L'état des services critiques (issu de Get-Service),
- L'identité de l'hôte (\$env:COMPUTERNAME).

Sans objet structuré, l'administrateur stocke chaque information dans des variables scalaires séparées :

```
$DisqueLibre = [math]::Round((Get-PSDrive -Name C).Free / 1GB, 1)  
$TopProcess = (Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First  
1).Name  
$ServiceSpool = (Get-Service -Name Spooler).Status
```

Le problème :

Ces variables vivent de façon déconnectée en mémoire. Si vous bouclez sur 20 serveurs :

- Impossible de trier le parc par serveur ayant le plus d'espace disque tout en affichant son service défaillant.
- Impossible de sérialiser le bilan d'un serveur sous forme d'une ligne unique cohérente.
- Les scripts deviennent vulnérables aux décalages d'index dans des tableaux parallèles.

La Solution avec [PSCustomObject] :

Chaque itération consolide atomiquement toutes les sondes dans une seule instance logique :

```
$Serveurs = @("SRV-AD01", "SRV-FILE01", "SRV-SQL01")  
  
$InventaireGlobal = foreach ($Machine in $Serveurs) {  
    # 1. Collecte des sondes indépendantes  
    $Disque = Get-PSDrive -Name C  
    $TopProc = Get-Process | Sort-Object WorkingSet64 -Descending | Select-Object -First 1  
    $Service = Get-Service -Name Spooler  
  
    # 2. Agrégation unifiée dans une entité métier
```

```
[PSCustomObject]@{
    Serveur      = $Machine
    EspaceLibreGo = [math]::Round($Disque.Free / 1GB, 1)
    TopProcessus = $TopProc.Name
    RAM_TopProcMo = [math]::Round($TopProc.WorkingSet64 / 1MB, 1)
    StatutSpooler = $Service.Status
    Horodatage    = (Get-Date -Format "HH:mm:ss")
}
}
```

La corrélation et le tri multi-critères deviennent triviaux :

```
$InventaireGlobal | Sort-Object EspaceLibreGo | Format-Table -AutoSize
```

L'intégrité face à la sérialisation (Background Jobs et Remoting)

Lorsque des données traversent la frontière d'un sous-processus (ou d'une liaison réseau WinRM, PowerShell convertit les objets en flux XML structuré (**sérialisation CLIXML**), puis les reconstruit à la réception (**désérialisation**).

- **Comportement d'un objet .NET complexe (ex: System.Diagnostics.Process) :**
Toutes ses méthodes actives (comme .Kill()) sont neutralisées. L'objet devient une coquille statique de type Deserialized.System.Diagnostics.Process.
- **Comportement d'un [PSCustomObject] :**
Un [PSCustomObject] est conçu comme un conteneur pur de propriétés métiers (NoteProperty). À travers un Job ou une session distante, ses propriétés, ses noms de colonnes et ses valeurs typées (entiers, flottants, chaînes, booléens) sont préservés à 100 %.

Le scénario d'échec (Objet .NET vivant à travers un Job) :

```
$Job = Start-Job -ScriptBlock { Get-Process -Name explorer }
$ProcessRecu = $Job | Receive-Job -Wait -AutoRemoveJob
```

```
$ProcessRecu.Kill() # ERREUR : La méthode n'existe plus sur l'objet désérialisé !
```

La Solution avec [PSCustomObject] :

On extrait les seules valeurs utiles dans un payload [PSCustomObject] à l'intérieur du Job :

APPROCHE INDUSTRIELLE : Payload de données pur et typé

```
$Job = Start-Job -ScriptBlock {
    $Proc = Get-Process -Name explorer
```

```

$OS = Get-CimInstance -ClassName Win32_OperatingSystem

[PSCustomObject]@{
    NomProcessus = $Proc.ProcessName
    PID          = $Proc.Id
    MemoireMo     = [math]::Round($Proc.WorkingSet64 / 1MB, 1)
    DernierReboot = $OS.LastBootUpTime
    DateAudit     = Get-Date
}
}

$Resultat = $Job | Receive-Job -Wait -AutoRemoveJob

# Les types .NET et les métriques sont conservés avec précision :
$Resultat.MemoireMo.GetType().FullName # System.Double (manipulable arithmétiquement)
$Resultat.DernierReboot.GetType().FullName # System.DateTime (soustractions possibles)

```

3. Anatomie et Avantages Comparatifs de [PSCustomObject]

La syntaxe littérale [PSCustomObject]@{} préfixe une table de hachage par un transtypageur de type .NET :

```

$MonServeur = [PSCustomObject]@{
    DateAudit = (Get-Date -Format "yyyy-MM-dd HH:mm")
    Serveur   = "SRV-FILE01"
    EspaceDisque = 42.5
    Statut    = "OPTIMAL"
}

```

3.1 Tableau comparatif approfondi des structures de restitution

Critère d'évaluation	Texte brut (Write-Host)	Chaîne formatée (String)	Table de Hachage (@{})	Objet Personnalisé ([PSCustomObject]@{})
Type .NET	Aucun (Flux 6)	System.String	System.Collections.Hashtable	System.Management.Automation.PSCustomObject
Capture en variable	Échec (\$null)	Récupère la chaîne	Récupère le dictionnaire	Récupère l'objet complet manipulable
Comportement pipeline	Inexistant	Traité comme un bloc texte	Unité associative unique	Chaque propriété devient une colonne exploitable
Ordre des propriétés	N/A	Fixé dans le texte	Aléatoire (hachage des clés)	Strictement garanti (ordre de déclaration)
Compatibilité Export-Csv	Échec	Ligne unique sans colonnes	Corrompu (métadonnées .NET)	Parfait (en-têtes et colonnes typées)
Compatibilité ConvertTo-Html	Échec	Tableau de longueurs de chaînes	Deux colonnes génériques Name/Value	Véritable tableau HTML avec colonnes dédiées
Tri (Sort-Object)	Impossible	Alphabétique brut	Complexe	Direct sur n'importe quelle propriété

5. Génération de Rapports Web avec ConvertTo-Html

La cmdlet ConvertTo-Html transforme une collection d'objets PowerShell en balisage web standardisé (<table>, <tr>, <th>, <td>).

5.1 Pourquoi le HTML brut est inexploitable ?

Par défaut, l'instruction suivante produit un tableau austère et illisible :

```
$BilanPoste | ConvertTo-Html | Out-File -FilePath "C:\Audit\rapport.html" -Encoding UTF8
```

Sans feuille de style CSS :

- Le tableau ne comporte aucune bordure visible,
- La police de caractères par défaut est *Times New Roman*,
- Les données textuelles et numériques sont collées aux bordures des cellules,
- Aucune distinction visuelle ne met en relief les alertes critiques.

5.2 Les paramètres majeurs de ConvertTo-Html

- **-Title** : Définit la balise <title> visible dans l'onglet du navigateur.
- **-Head** : Permet d'injecter du balisage d'en-tête, notamment les styles <style> ... </style> et les balises <meta>.
- **-PreContent** : Ajoute du balisage HTML (titres <h1>, bannières) immédiatement avant le tableau.
- **-PostContent** : Ajoute du balisage (mentions de copyright, heure de génération) juste après le tableau.
- **-Fragment** : Émet exclusivement le code du tableau (<table>...</table>), sans l'encapsulation structurelle <html>, <head> et <body>. C'est le paramètre indispensable pour concevoir des rapports multi-sections.

6. Stylisation CSS Embarquée pour Tableau de Bord

Pour garantir qu'un rapport HTML reste entièrement autonome (consultable hors-ligne, transférable par email sous forme de fichier unique sans dépendance externe), la feuille de style CSS doit être **embarquée directement dans l'en-tête HTML**.

6.1 Modèle de feuille de style moderne et responsive

```
$StyleCSS = @"
```

```
<style>
```

```
body {
```

```
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
```

```
    background-color: #f4f6f9;
```

```
    color: #2c3e50;
```

```
    margin: 25px;
```

```

}
h1 {
  color: #1a365d;
  border-bottom: 3px solid #3182ce;
  padding-bottom: 10px;
  font-size: 1.8em;
}
h2 {
  color: #2b6cb0;
  margin-top: 25px;
  font-size: 1.3em;
}
.horodatage {
  font-size: 0.85em;
  color: #718096;
  margin-bottom: 20px;
}
table {
  width: 100%;
  border-collapse: collapse;
  margin-top: 15px;
  background-color: #ffffff;
  box-shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.1);
  border-radius: 6px;
  overflow: hidden;
}
th {
  background-color: #2b6cb0;
  color: #ffffff;
  text-align: left;
  padding: 12px 15px;
  font-size: 0.9em;
  text-transform: uppercase;
  letter-spacing: 0.5px;
}
td {
  padding: 10px 15px;
  border-bottom: 1px solid #e2e8f0;
  font-size: 0.9em;
}
tr:nth-child(even) {
  background-color: #f7fafc;
}

```

```

tr:hover {
    background-color: #edf2f7;
}
/* Badges d'état */
.badge-optimal {
    color: #22543d;
    background-color: #c6f6d5;
    padding: 4px 10px;
    border-radius: 4px;
    font-weight: 600;
}
.badge-critique {
    color: #742a2a;
    background-color: #fed7d7;
    padding: 4px 10px;
    border-radius: 4px;
    font-weight: 600;
}
</style>
"@

```

7. L'Approche par Fragments : Rapports Multi-Sections (-Fragment)

Dans une infrastructure de production, un tableau de bord global ne se résume pas à une table unique. Il regroupe souvent plusieurs domaines distincts :

1. Une section de santé des disques,
2. Une section d'inventaire des services essentiels,
3. Une section des processus consommateurs de ressources.

Si l'on appelle `ConvertTo-Html` successivement sur chaque collection, PowerShell génère des documents HTML complets successifs comprenant chacun des balises `<html>` et `<body>`, ce qui produit un document non conforme et visuellement corrompu.

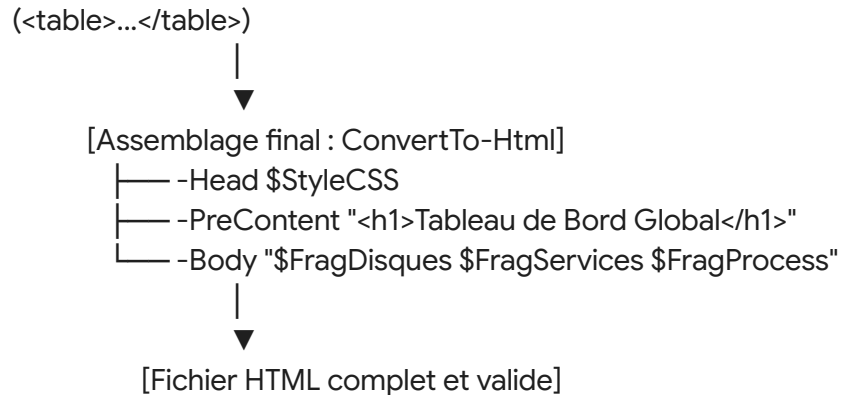
7.1 Patron d'assemblage modulaire

1. Chaque collection d'objets est convertie indépendamment avec le paramètre **-Fragment**,
2. Les fragments tabulaires sont assemblés au sein d'une variable de corps global,
3. L'ensemble est encapsulé dans un appel final à `ConvertTo-Html` :

```

[Collection Disques] —> ConvertTo-Html -Fragment —> $FragDisques (<table>...</table>)
[Collection Services] —> ConvertTo-Html -Fragment —> $FragServices (<table>...</table>)
[Collection Top Proc] —> ConvertTo-Html -Fragment —> $FragProcess

```



7.2 Implémentation du patron par fragments

1. Génération des fragments HTML indépendants

```
$FragDisques = $DonneesDisques | ConvertTo-Html -Fragment -PreContent "<h2>État du
Stockage</h2>"
```

```
$FragServices = $DonneesServices | ConvertTo-Html -Fragment -PreContent "<h2>Services
Stratégiques</h2>"
```

2. Concaténation des fragments dans un conteneur unique

```
$CorpsGlobal = @"
```

```
  $FragDisques
```

```
  <br/>
```

```
  $FragServices
```

```
"@"
```

3. Encapsulation globale dans la page finale

```
$RapportFinal = ConvertTo-Html -Title "Supervision d'Infrastructure" `
```

```
  -Head $StyleCSS `
```

```
  -PreContent "<h1>Tableau de Bord de Production</h1><p
```

```
class='horodatage'>Généré le $(Get-Date)</p>" `
```

```
  -Body $CorpsGlobal
```

8. Coloration Conditionnelle et Post-Traitement Dynamique

Par conception, la cmdlet `ConvertTo-Html` génère des balises `<td>` simples et ne dispose d'aucun paramètre interne pour appliquer des classes CSS dynamiques basées sur les valeurs des cellules.

Pour transformer des mentions textuelles brutes en badges colorés ou en lignes d'alerte, la

technique standard consiste à appliquer une **post-substitution ciblée via l'opérateur -replace**.

8.1 La technique de substitution de cellule

Après la génération de la chaîne HTML par ConvertTo-Html, on intercepte les mots-clés d'état pour les envelopper dans des conteneurs dotés de classes CSS :

1. Génération du code HTML brut

```
$CodeHtml = $DonneesAudit | ConvertTo-Html -Head $StyleCSS -Title "Contrôle Parc"
```

2. Post-substitution des cellules de statut par des balises avec classes CSS

```
$CodeHtmlFinal = $CodeHtml `
    -replace '<td>OPTIMAL</td>', '<td><span class="badge-optimal">OPTIMAL</span></td>' `
    -replace '<td>CRITIQUE</td>', '<td><span class="badge-critique">CRITIQUE</span></td>' `
    -replace '<td>ALERTE</td>', '<td><span class="badge-critique">ALERTE</span></td>'`
```

3. Écriture du fichier final

```
$CodeHtmlFinal | Out-File -FilePath "C:\Audit\dashboard.html" -Encoding UTF8
```

8.2 Coloration d'une rangée complète (<tr>)

Pour accentuer la criticité d'un incident, il est souvent souhaitable de surligner la ligne entière du tableau. On utilise alors une expression régulière exploitant une assertion avant positive (*lookahead* (?=...)) :

Détection de la présence de "CRITIQUE" dans une cellule de la même rangée

```
$CodeHtmlFinal = $CodeHtmlFinal -replace '<tr>(?!<td>.*?CRITIQUE.*?</td>)', '<tr style="background-color: #fff5f5;">'`
```

9. Synthèse des Bonnes Pratiques de Production

1. **Bannir Write-Host pour les données métiers** : Renvoyer exclusivement des objets [PSCustomObject]. La présentation console ne doit être qu'un formatage secondaire de l'objet via Format-Table ou Out-GridView.
2. **Respecter la cohérence des colonnes** : Dans une boucle ou une collection de [PSCustomObject], veillez à ce que chaque objet possède exactement les mêmes noms de clés et le même ordre de déclaration.
3. **Forcer systématiquement l'encodage UTF-8** : Lors de l'écriture du fichier HTML via Out-File -Encoding UTF8, afin de garantir la restitution exacte des caractères accentués (*Mémoire, Arrêté, Système*).
4. **Découpler calcul et présentation** : Réalisez l'ensemble des calculs arithmétiques, arrondis et arbitrages logiques lors de la création de l'objet. Le code HTML ne doit faire

que restituer l'état sans recalculer.

5. **Embarquer les styles CSS** : Intégrez systématiquement les règles CSS dans le bloc `<style>` passé à -Head pour garantir que le rapport reste consultable hors connexion ou par email sans dépendance à une feuille de style externe.